Building P2 from Zero — Checklist

CS 6983 — Week 8 Handout

CS 6983 · Week 8 · Companion to the Week 8 slides, "Building P2 from Zero."

Work through these steps in order with your partner, and tick each item off in your own repo. Every item applies to any P2 app. Where the worked example, vis-paper-planner, already shows a step, there is a **See it** link.

This checklist gets you started. It is not the rubric. See "What P2 also requires" at the end, and the P2 spec.

Steps 0–1: Idea, research, PRD

- [ ] Brainstorm 2–3 ideas with Claude Web and pick one a real user would want (Weeks 4–5)
- [ ] *Optional for P2:* hold 3+ Mom Test conversations and keep the notes in `docs/research/`
- [ ] Write `docs/PRD.md`: problem, personas, user stories, non-goals
- [ ] Give every user story acceptance criteria you could write a test for
- [ ] Pick your stack from the P2 list: React/Next.js + Tailwind, Express or Next.js API routes, PostgreSQL or MongoDB, JWT or OAuth

See it: PRD · Mom Test summary

Step 2: DESIGN.md

- [ ] Put a mockup in `docs/design/`: a Claude Web artifact, Figma export, or sketch
- [ ] Add `DESIGN.md` at the repo root, with tokens (colors, type, spacing) and sections from Overview to Do's and Don'ts
- [ ] `npx @google/design.md lint DESIGN.md` passes
- [ ] Generate the Tailwind theme from it (`npx @google/design.md export --format css-tailwind DESIGN.md`) instead of copying values by hand
- [ ] `AGENTS.md` says "UI work follows `DESIGN.md`"

Format: Google Labs DESIGN.md spec. It's an alpha, open-source format. It loads into the AI's context like any other file, so keep it short.

See it: mockup

Step 3: Rules and memory files

- [ ] Write one `AGENTS.md` at the repo root together. Don't copy it into tool folders.
- [ ] It covers stack and versions, exact commands, conventions, the testing rule, security don'ts, and where docs live
- [ ] It records your workflow: Product Owner and Scrum Master rotate each sprint, one branch per issue, partner review before merge
- [ ] It stays under ~200 lines and links to `docs/` instead of pasting it
- [ ] Tool folders (`.agents/`, `.cursor/`, `.claude/`) hold only path-scoped rules or skills
- [ ] Change it through PRs, and revisit it at every retro
- [ ] When you start Claude Code (Week 9), add a `CLAUDE.md` containing `@AGENTS.md`

`AGENTS.md` counts as your shared rules file for HW3 and P2. Older names such as `.antigravityrules` are still accepted.

See it: AGENTS.md. The content is good; today you'd drop its symlink and its Antigravity frontmatter.

Step 4: Backlog and board

- [ ] Turn user stories into GitHub issues using an issue template (story, acceptance criteria, estimate)
- [ ] Create one milestone per sprint, and the labels `feature`, `bug`, `chore`
- [ ] Set board columns: Todo · In Progress · In Review · Done
- [ ] Write `docs/sprints/sprint-1/planning.md`: goal, issues, estimates, who is PO and who is SM
- [ ] Add a PR template with `Closes #N`, how it was tested, and a partner-review checkbox
- [ ] For each issue: branch → implement in Agent mode → self-review with AI → PR → partner review → merge → Done

See it: Sprint 1 milestone and issues

Step 5: CI

- [ ] `.github/workflows/ci.yml` runs on every PR
- [ ] It runs in stages: lint + format → unit + integration → build → E2E (Playwright)
- [ ] It has a security-scan job (for example `npm audit`, CodeQL)
- [ ] PRs merge only when CI is green (branch protection)
- [ ] A pre-commit hook runs lint and format locally (Husky + lint-staged)
- [ ] CI never uses production secrets; tests hit a local or test database

See it: pre-commit hook

Step 6: First feature, test-first

- [ ] Pick a thin vertical slice: one page → one API route → one table
- [ ] Commit the failing test first (red), then the code (green), then refactor
- [ ] Keep the schema in a migration file, not clicked together in a dashboard
- [ ] Write a seed script so your partner and CI get the same data
- [ ] Test the error paths: empty input, bad input, database down
- [ ] Document the endpoint in `docs/api/openapi.yaml`
- [ ] Check the page against the mockup in browser mode; use a database MCP server to read the real schema

Step 7: Deploy

- [ ] Production deploys from `main`, and every PR gets a preview deploy
- [ ] Env vars are set per environment in the host; `.env.example` lists names only
- [ ] Running database migrations against production is a written-down step
- [ ] Previews use a test database (or you know they share production data)
- [ ] One health-check URL proves the app *and* the database are up
- [ ] The README has the live URL, how to run locally, and the deploy steps

Deploy the empty starter on day one, so deploying is never the risky last step.

You have	Options
Next.js, with API routes	Vercel, Netlify

A separate Express server	Render
PostgreSQL	Supabase (free projects can be paused)
MongoDB	MongoDB Atlas

Pitfalls

- **A `CLAUDE.md` turns off `AGENTS.md` in Claude Code.** Claude reads `AGENTS.md` only when there is no `CLAUDE.md`, and a personal `CLAUDE.local.md` counts too. Fix: make `CLAUDE.md` say `@AGENTS.md` (Claude Code memory docs).
- **Rules copied into several tool folders.** They get loaded twice, then drift apart.
- **Memory files that keep growing.** Longer files use more context and get followed less closely.
- **Nested `AGENTS.md` files.** VS Code Copilot ignores them by default, so keep rules at the root.
- **A brand's DESIGN.md copied from a gallery.** Your app ends up looking like someone else's.
- **A committed `.env`.** Deleting it isn't enough; rotate the keys.

What P2 also requires

Area	Required
Features	Auth (JWT or OAuth) · 3+ features · real-time updates or complex state · a public API
Testing	Unit + integration + E2E, failing tests committed first
CI/CD	Multi-stage pipeline · deploy previews · security scanning · automated deploy
Process	2+ sprints · planning + retros · 3+ standups per partner per sprint · 5+ PR reviews per partner · estimates
Docs	README · OpenAPI docs · architecture + DB-schema diagrams · setup guide · 10-min demo video · contribution log · 300-word reflection each